

VISHAL JATIN SONEJI

Los Angeles, CA | +1 (213) 827-4923 | vishalsoneji21@gmail.com | [LinkedIn](#) | [GitHub](#) | [Portfolio](#)

PROFESSIONAL SUMMARY

Gameplay Programmer with a shipped Steam title and engine-level C++ systems experience across custom and commercial engines. Skilled in UE5 multiplayer replication, networking, Custom Depth rendering, and DirectX 11 GPU pipeline programming. Targeting Gameplay or Game Engine Engineer roles at studios building multiplayer and systems-heavy titles.

EDUCATION

University of Southern California

Master of Science in Computer Science · GPA: 3.5/4.0

Relevant Coursework : Game Development, 3D Rendering & Graphics, Advanced Algorithms

Aug 2024 – May 2026

Los Angeles, CA

Charotar University of Science and Technology

Bachelor of Technology in Computer Science · CGPA: 9.32 / 10

Relevant Coursework : AR/VR, Game Development

Oct 2020 – May 2024

Anand, Gujarat

WORK EXPERIENCE

Gameplay Programmer | Lucky Duckies

May 2025 – May 2026

USC Games

Los Angeles, CA

- Collaborated as a **Full-stack gameplay programmer** on **Lucky Duckies**, co-op adventure game released publicly on **Steam**, contributing across multiplayer networking, player systems, visual polish, and performance optimization
- Architected a **split-screen co-op** system in **C++** on a listen-server model in **Unreal Engine 5**, partitioning a 1920×1080 viewport into two independent 960×1080 Player-Controller cameras at runtime with dedicated HUD layers per player, enabling seamless **local + online co-op** as a core feature
- Built a player **occlusion** silhouette system using **Custom Depth stencil** and a hand-authored **Post-Process Material**, rendering a colored outline whenever player is hidden behind geometry improved player spatial awareness and tracking feedback, with **70% of players** responding positively to the feature across all 6 levels
- Engineered a skeletal socket and bone attachment framework for cosmetic **skin system**, authoring 5 sockets (head, neck, torso, left/right wing) in **Skeleton Editor** so all accessories animate correctly with player movement at runtime, creating a deployable cosmetic progression layer
- Led **lighting optimization** across all **6 levels** in a mixed **static/Lumen** pipeline, auditing **100+ lights** via **Optimization View modes** (Lighting Complexity, Lightmap Density, Shader Complexity) to eliminate GPU overdraw, improving frame rate from an unstable **50fps** to a consistent **90–120fps** on target hardware
- Resolved **15+ cross-discipline gameplay bugs** including a checkpoint/respawn regression where players spawned underground was fixed by replacing random sphere-offset logic with a deterministic world-space height clamp against level collision

PROJECTS

Frustum Culling

- Generated a **6-plane frustum culling** system in **C++**, computing plane normals via cross products and per-plane signed distances via dot products to establish per-mesh AABBs at load time eliminating draw call submissions for all off-screen geometry and achieving a **3 times fps improvement (100→300fps)** across **1,000-mesh** stress test scenes
- Engineered a scene-wide wireframe **AABB debug renderer** with a **100-meter draw distance** cap reflecting LOD boundaries, overlaying all active bounding volumes simultaneously for real-time culling validation profiled across **5 scenes** with **500+ meshes** each to verify GPU draw call reduction

Skeletal Animation Blending

- Designed an **8-slot skeletal animation system** in **C++** with per-slot weight, looping, and frame-normalized playback managing clean state transitions via event callbacks across full-body, partial-body, and additive blend modes
- Orchestrated per-joint quaternion **SLERP** blending across a **30-joint** humanoid hierarchy, sampling **TSQ keyframes** each frame to produce smooth, gimbal-lock-free character motion uploaded to the GPU
- Built **additive and partial-body blend modes** with independent upper/lower joint chain masking, layering up to 3 animations on a single character state and reducing required animation assets from **12 to 7**

Rigid Body Physics System

- Introduced **collision detection** and **gravity** to a custom **C++ engine** that previously had none preventing character-geometry clipping across 3 object types (soldier, car, floor) and unlocking gameplay systems like bullet damage that are impossible without **per-object collision volumes**
- Implemented **AABB-based collision** across **3 primitive types (Sphere, Box, Plane)** with **3 collision responses (stop, slide, freefall)** leveraging inverse world transform projection eliminating edge-case misses for deterministic hit detection
- Designed **Physics Component** to auto-generate a fitted **AABB** from any object's own dimensions reducing physics integration for any new object from custom per-object collision code down to **1 function call** with zero changes to existing code

GPU Particle System

- Architected a **GPU-driven particle system** deploying a **3-pass DirectX 11 Compute Shader pipeline** (emit → update → render) with **ping-pong UAV buffers**, facilitating fire, smoke, explosion, and rain driven by **HLSL** constant buffers updated each frame
- Diagnosed and fixed 5 critical pipeline bugs by identifying broken **CPU/GPU buffer** handoff and a missing update stub restoring stable simulation across **5-6 emitters** at **200-300 particles** each, scaling to **10,000 particles** on a single emitter with no performance degradation
- Mobilized a movable **Particle Emitter Scene Node** integrated into engine's scene graph, driving per-particle spawn rate, velocity, lifetime, and color via **HLSL constant buffers**

TECHNICAL SKILLS

Languages: C++, Python, HLSL, Blueprint Visual Scripting

Engines & APIs: Unreal Engine 5 (UE5), Prime Engine (custom C++), Unity, DirectX 11

Engine Systems: Actor Replication, Server/Client RPCs, Listen-Server, Custom Depth / Post-Process, Lumen, Lightmaps, Frustum Culling, Skeletal Mesh / Bone Sockets, Quaternion SLERP, Rigid Body Physics, GPU Particle Systems, Gameplay Framework, Animation Blueprints, UMG, Viewport Management

Tools & Process: Perforce, Git, RenderDoc, Unreal Insights, Blender, Maya, Agile / Scrum